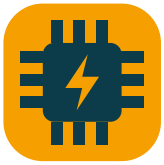
INSTITUTO POLITÉCNICO NACIONAL

Unidad Profesional Interdisciplinaria en Ingeniería y Tecnologías Avanzadas (UPIITA)

Ingeniería Telemática



ElectroKit

Aplicación web de catálogo de productos con autenticación validada en backend

Documentación de diseño · Extraordinario

Alumna:	Atziry Vianey Martínez Navarro
Tecnologías:	HTML5 · CSS3 · JavaScript · Node.js + Express · MySQL
Fecha:	Septiembre de 2026

Contenido

1. Introducción y requerimientos
2. Arquitectura del sistema
3. Modelado UML (casos de uso, clases, secuencia y actividad)
4. Maquetado y diagrama de navegación
5. Modelo de datos
6. Pruebas de funcionamiento
7. Instalación y ejecución

1. Introducción y requerimientos

ElectroKit es una aplicación web para una tienda de componentes electrónicos. El usuario inicia sesión mediante un formulario HTML validado con JavaScript; el servidor web recibe el usuario y la contraseña, los asigna a variables en el backend, consulta al gestor de base de datos y compara la contraseña almacenada con la enviada. Según el resultado, el servidor responde con una página que indica si la autenticación fue **correcta** o **errónea**. Una vez autenticado, el usuario accede a un catálogo presentado en una **tabla HTML de 3 columnas × 3 filas** (9 productos).

1.1 Requerimientos funcionales

ID	Requerimiento
RF-01	Mostrar una página de login con campos de usuario y contraseña.
RF-02	Validar en el navegador (JavaScript) que los campos no estén vacíos y cumplan el formato antes de enviarlos.
RF-03	El servidor web asigna el usuario y la contraseña recibidos a variables del backend.
RF-04	El backend consulta la base de datos buscando al usuario y obtiene su contraseña.
RF-05	El backend compara la contraseña de la BD con la enviada desde el navegador.
RF-06	Responder al navegador con una página de “Autenticación correcta” o “Autenticación errónea”.
RF-07	Mostrar los productos en una tabla HTML de 3 columnas y 3 filas.
RF-08	Por producto: nombre, descripción y fecha de envío estimada.
RF-09	Por producto: calificación con imágenes de estrellas rellenas y vacías.
RF-10	Por producto: una imagen grande y 5 miniaturas que se intercambian con la imagen grande al hacer clic.
RF-11	Por producto: botón “Comentarios” que despliega la lista de comentarios de usuarios y un formulario para agregar uno nuevo.

1.2 Requerimientos no funcionales

ID	Requerimiento
RNF-01	Una sola tabla en el gestor de base de datos (`usuarios`) con los datos de usuario y password.
RNF-02	Consultas parametrizadas para evitar inyección SQL.
RNF-03	Validación de formato duplicada en navegador y servidor (el servidor no confía en el cliente).
RNF-04	Diseño responsivo: en pantallas menores a 900 px cada celda de la tabla ocupa todo el ancho.
RNF-05	El catálogo solo es accesible con sesión iniciada (cookie firmada).

2. Arquitectura del sistema

El sistema sigue una **arquitectura cliente–servidor de tres capas**. El navegador (capa de presentación) solicita páginas y envía el formulario de login por HTTP al **servidor web** Node.js + Express (capa de lógica), el cual se conecta por TCP (puerto 3306) al **gestor de base de datos** MySQL (capa de datos) mediante el controlador `mysql2`.

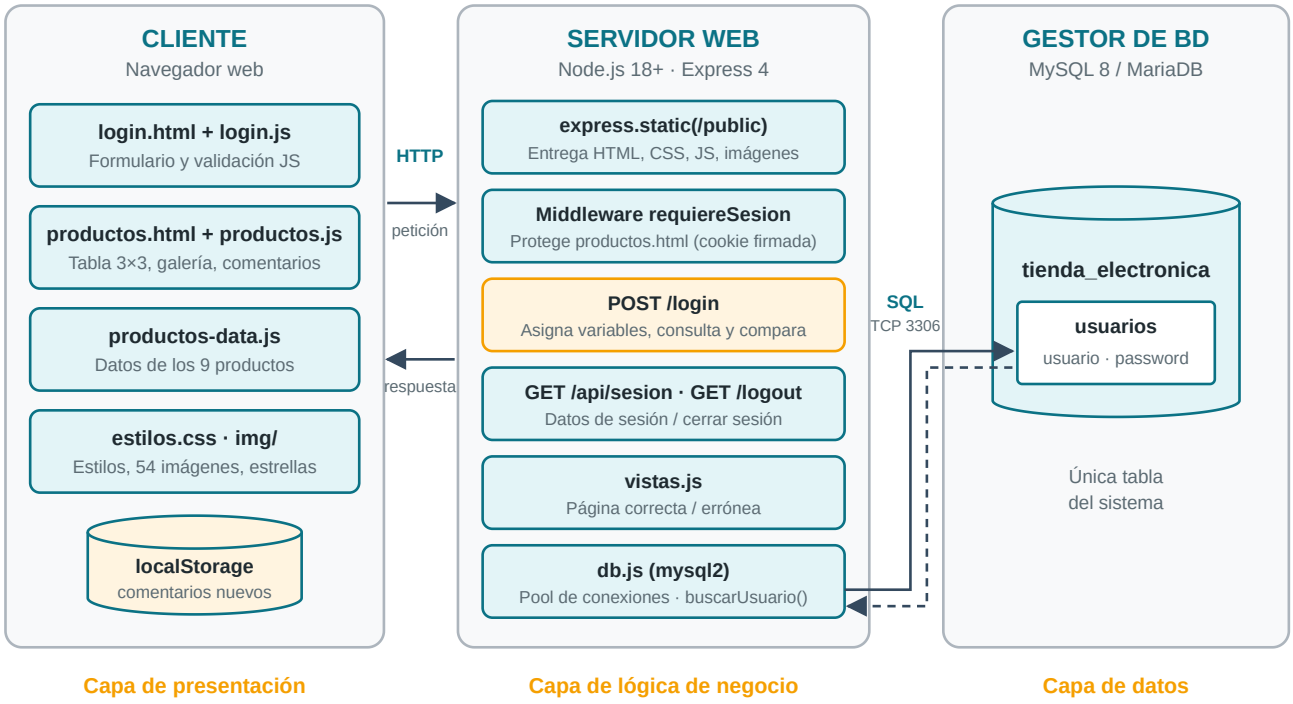


Figura 1. Arquitectura de tres capas del sistema ElectroKit.

2.1 Tecnologías

Capa	Tecnología	Función
Presentación	HTML5, CSS3, JavaScript (ES6)	Formularios, tabla 3×3, validación en cliente, galería y comentarios.
Servidor web	Node.js 18+, Express 4	Servir archivos estáticos, rutas `/login`, `/api/sesion`, `/logout`.
Middleware	express.urlencoded, cookie-parser	Leer el cuerpo del formulario y manejar la cookie de sesión firmada.
Acceso a datos	mysql2/promise (pool)	Consultas parametrizadas asíncronas a MySQL.
Base de datos	MySQL 8 / MariaDB 10+	Tabla `usuarios`.

2.2 Estructura del proyecto

```
tienda-electronica/
├── server.js              Servidor web: rutas, middleware, validación de login
├── db.js                  Conexión (pool) a MySQL y función buscarUsuario()
├── vistas.js              Páginas HTML de respuesta (correcta / errónea)
├── database.sql           Script: base de datos y tabla usuarios
├── package.json
└── public/
    ├── login.html         Página de inicio de sesión
    ├── productos.html     Catálogo (tabla 3×3)
    ├── css/estilos.css
    ├── js/login.js        Validación del formulario
    ├── js/productos-data.js  Datos de los 9 productos
    ├── js/productos.js    Tabla, estrellas, galería, comentarios
    └── img/               logo, estrellas llena/vacía, 54 imágenes de producto
```

2.3 Interfaz HTTP (rutas del servidor)

Método	Ruta	Descripción	Respuesta
GET	`/`	Redirige a la página de login.	302 → /login.html
GET	`/login.html`	Formulario de acceso (archivo estático).	200 HTML
POST	`/login`	Recibe `usuario` y `password`, consulta la BD y compara.	200 correcta · 401 errónea · 400 formato · 500 BD
GET	`/productos.html`	Catálogo; protegido por el middleware `requiereSesion`.	200 HTML o 302 → login
GET	`/api/sesion`	Devuelve usuario y nombre de la sesión.	200 JSON · 401
GET	`/logout`	Elimina la cookie de sesión.	302 → /login.html

2.4 Validación en el backend

El siguiente fragmento de `server.js` implementa el requerimiento central: asignar los valores recibidos a variables, consultar la base de datos y comparar la contraseña.

```
// ---------- POST /login : validación de usuario y password en backend ----------
app.post('/login', async (req, res) => {
  // 1. Asignar los valores recibidos del navegador a variables del backend
  const usuario = (req.body.usuario || '').trim();
  const password = req.body.password || '';

  // 2. Validar formato
  if (!REGEX_USUARIO.test(usuario) || !REGEX_PASSWORD.test(password)) {
    return res.status(400).send(paginaError('Los datos enviados no tienen un formato válido.'));
  }

  try {
    // 3. Petición a la base de datos buscando la información del usuario
    const registro = await buscarUsuario(usuario);

    // 4. Comparar el password obtenido de la BD con el enviado desde el navegador
    if (registro && registro.password === password) {
      res.cookie('usuario', registro.usuario, { signed: true, httpOnly: true, sameSite: 'lax' });
      res.cookie('nombre', registro.nombre, { signed: true, httpOnly: true, sameSite: 'lax' });
      return res.send(paginaExito(registro.nombre, registro.usuario));
    }

    // 5. Responder con página de error (no se revela si falló usuario o password)
    return res.status(401).send(paginaError('Usuario o contraseña incorrectos.'));
  } catch (err) {
    console.error('Error de base de datos:', err.message);
    return res.status(500).send(paginaError('No fue posible conectar con la base de datos. Intenta
más tarde.'));
  }
});
```

```
/**
 * Busca un usuario por su nombre de usuario.
 * Devuelve { usuario, password, nombre } o null si no existe.
 */
async function buscarUsuario(usuario) {
  const sql = 'SELECT usuario, password, nombre FROM usuarios WHERE usuario = ? LIMIT 1';
  const [filas] = await pool.execute(sql, [usuario]); // consulta parametrizada (evita inyección
SQL)
  return filas.length ? filas[0] : null;
}
```

3. Modelado UML

3.1 Diagrama de casos de uso

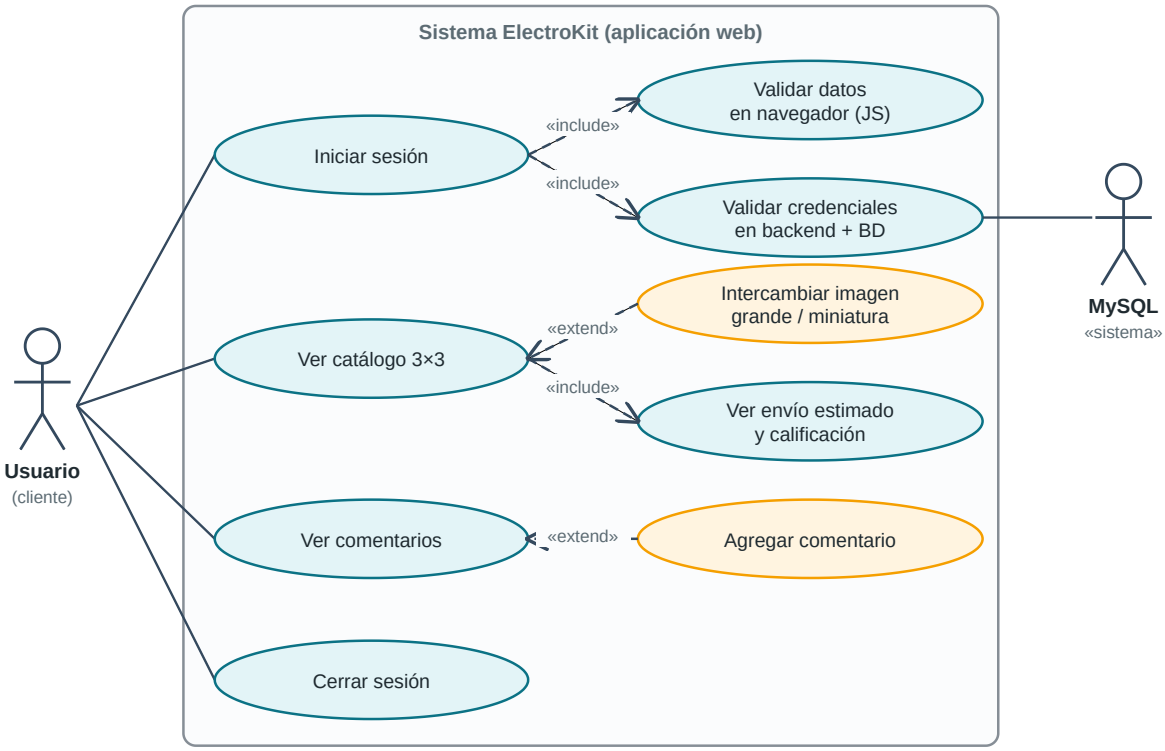


Figura 2. Casos de uso del sistema. El actor secundario MySQL participa en la validación de credenciales.

Especificación: CU-01 Iniciar sesión	
Actor principal	Usuario. Actor secundario: gestor de BD MySQL.
Precondición	El usuario está registrado en la tabla `usuarios`.
Flujo principal	1. El usuario abre login.html. 2. Captura usuario y contraseña. 3. El navegador valida el formato. 4. Se envía POST /login. 5. El servidor asigna los valores a variables y valida el formato. 6. Consulta la BD por el usuario. 7. Compara la contraseña obtenida con la enviada. 8. Responde “Autenticación correcta” y crea la sesión.
Flujos alternos	3a. Formato inválido: se muestran mensajes y no se envía. 6a. Usuario inexistente o 7a. contraseña distinta: se responde “Autenticación errónea”. 6b. BD no disponible: página de error 500.
Postcondición	Cookie de sesión firmada; acceso al catálogo.

Caso de uso	Descripción breve
CU-02 Ver catálogo 3×3	Muestra 9 productos en una tabla de 3 filas × 3 columnas con nombre, descripción, fecha de envío estimada y calificación.
CU-03 Intercambiar imagen	Al hacer clic (o Enter) en una miniatura, esta toma el lugar de la imagen grande y viceversa.
CU-04 Ver comentarios	El botón “Comentarios (n)” despliega la lista con usuario, estrellas, fecha y texto.

CU-05 Agregar comentario	El usuario elige de 1 a 5 estrellas y escribe un texto (mín. 3 caracteres); se agrega a la lista y se recalcula la calificación.
CU-06 Cerrar sesión	Elimina la cookie y regresa al login.

3.2 Diagrama de clases

Como el sistema está escrito en JavaScript por módulos, cada módulo se modela como una clase con el estereotipo del archivo que la implementa. `Usuario` corresponde al registro de la tabla de la base de datos; `Producto` y `Comentario` son estructuras del lado del cliente.

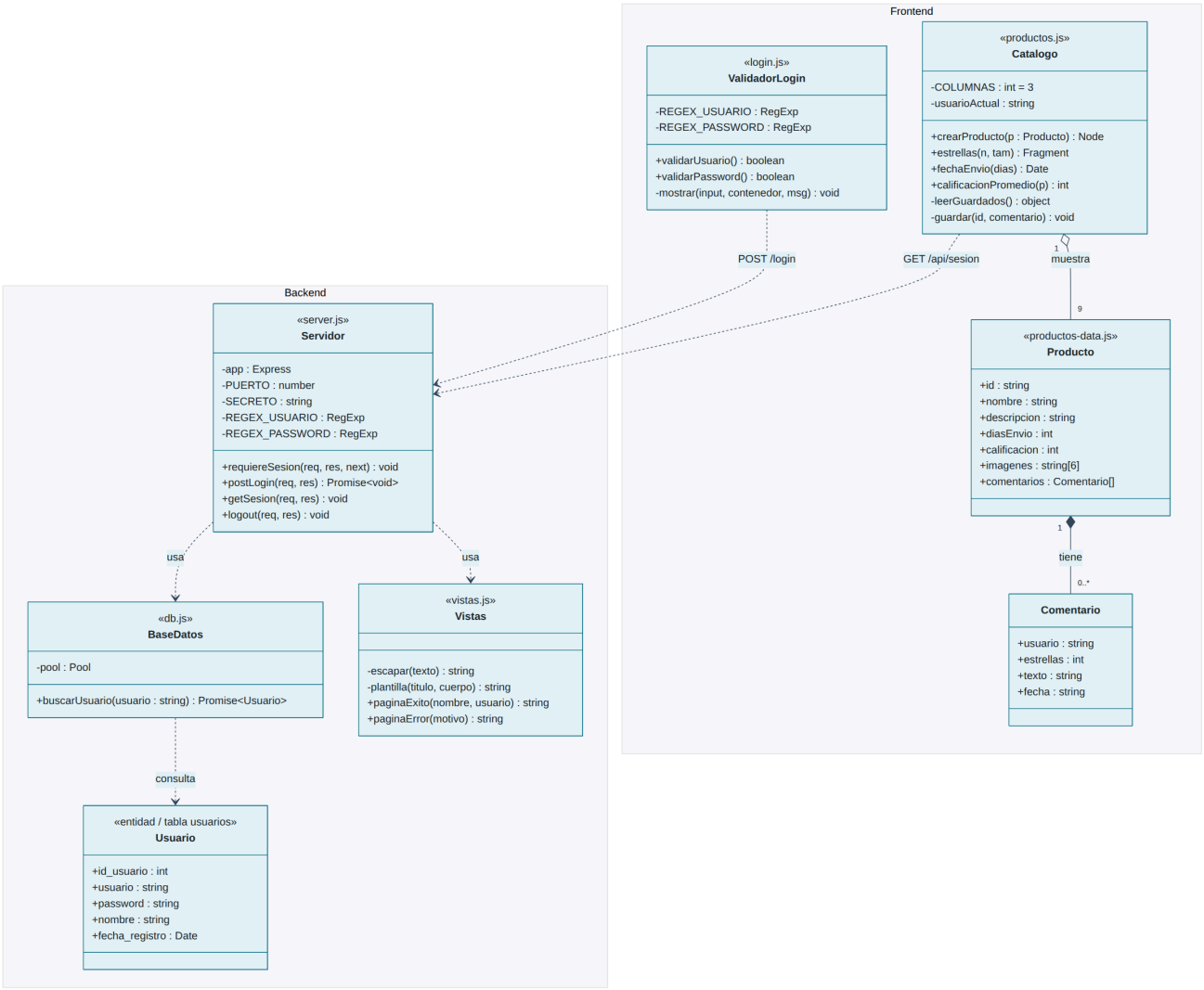


Figura 3. Diagrama de clases (backend y frontend).

3.3 Diagrama de secuencia — Validación de login

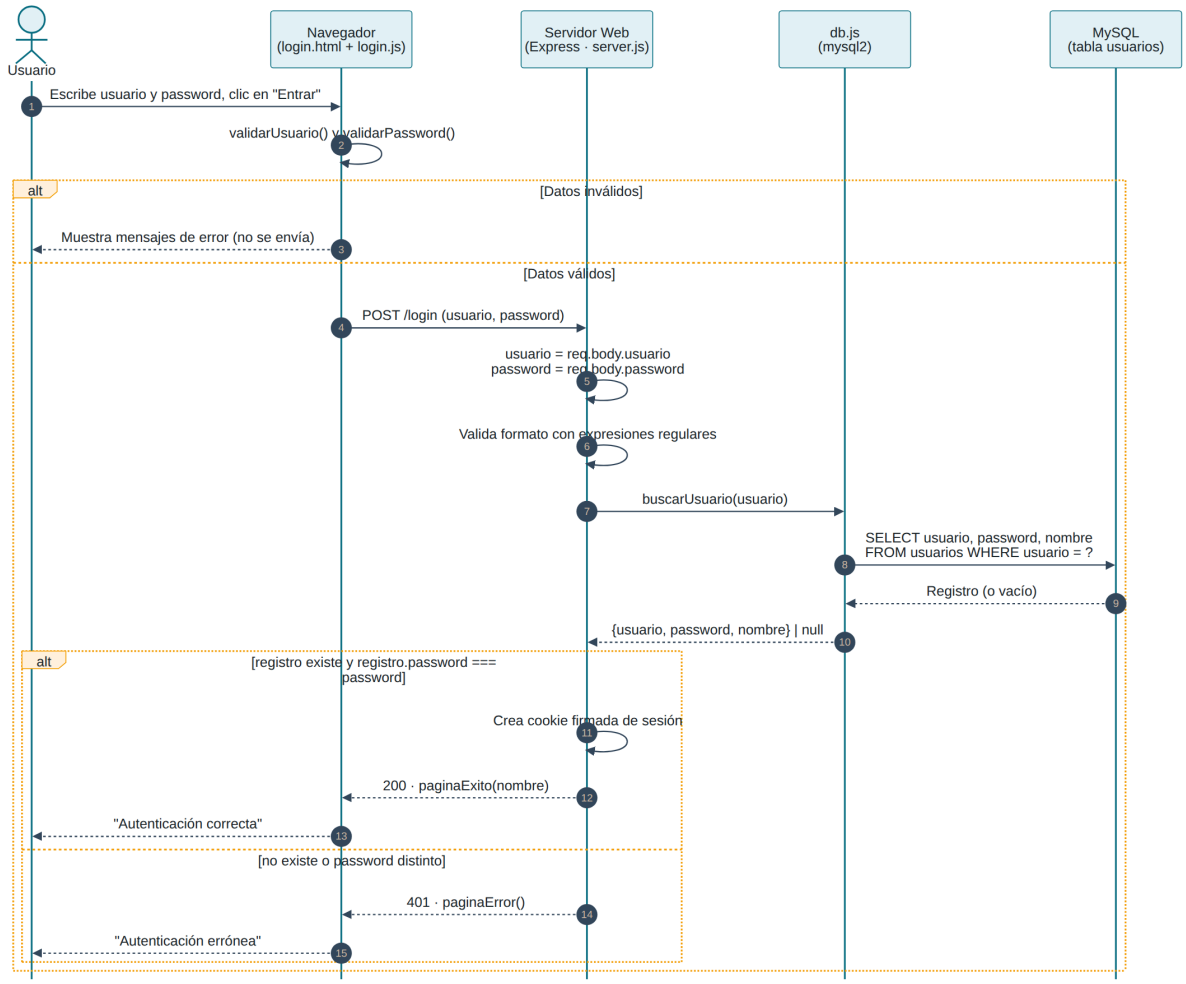


Figura 4. Secuencia del inicio de sesión: navegador → servidor web → base de datos → página de respuesta.

3.4 Diagrama de secuencia — Uso del catálogo

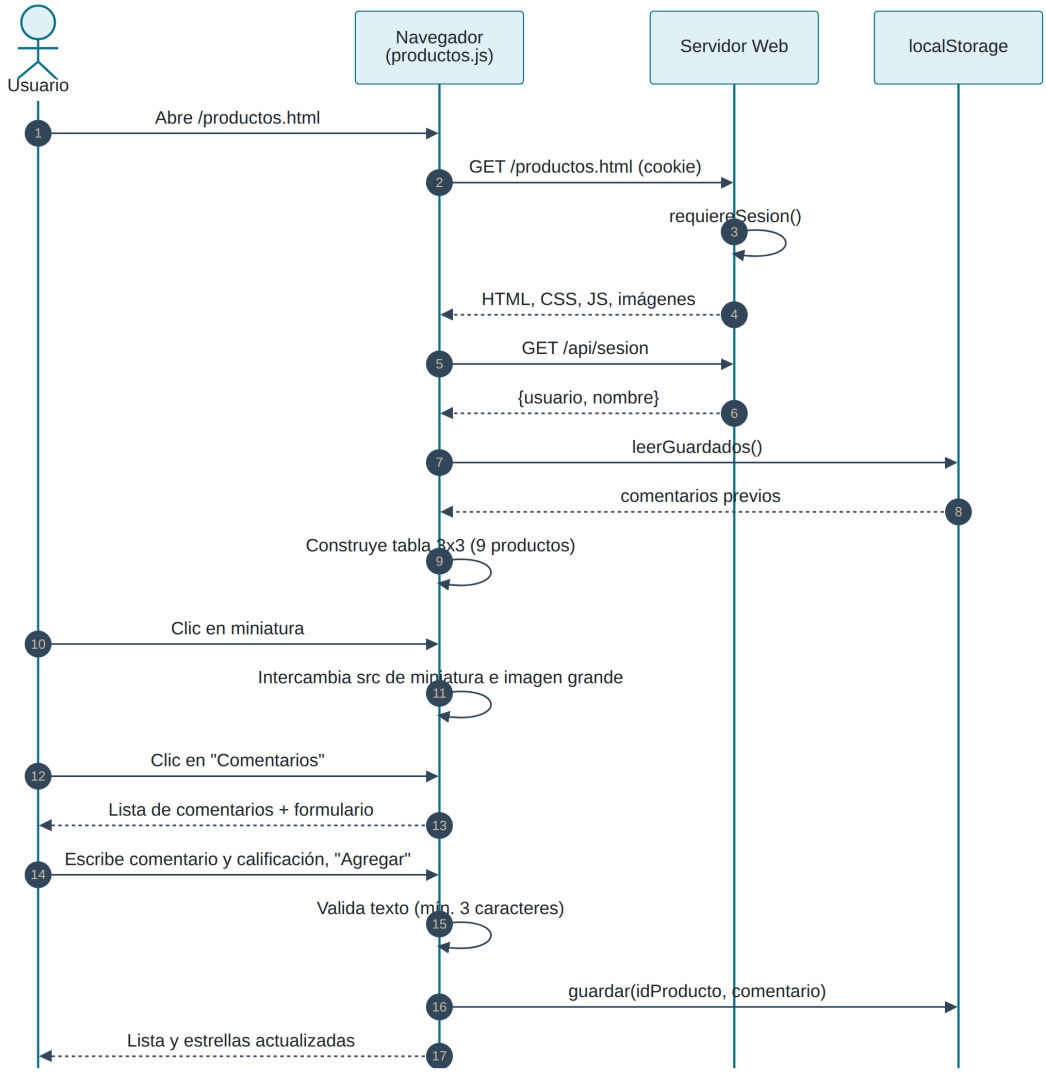


Figura 5. Carga del catálogo, intercambio de imágenes y alta de un comentario.

3.5 Diagrama de actividad — Autenticación

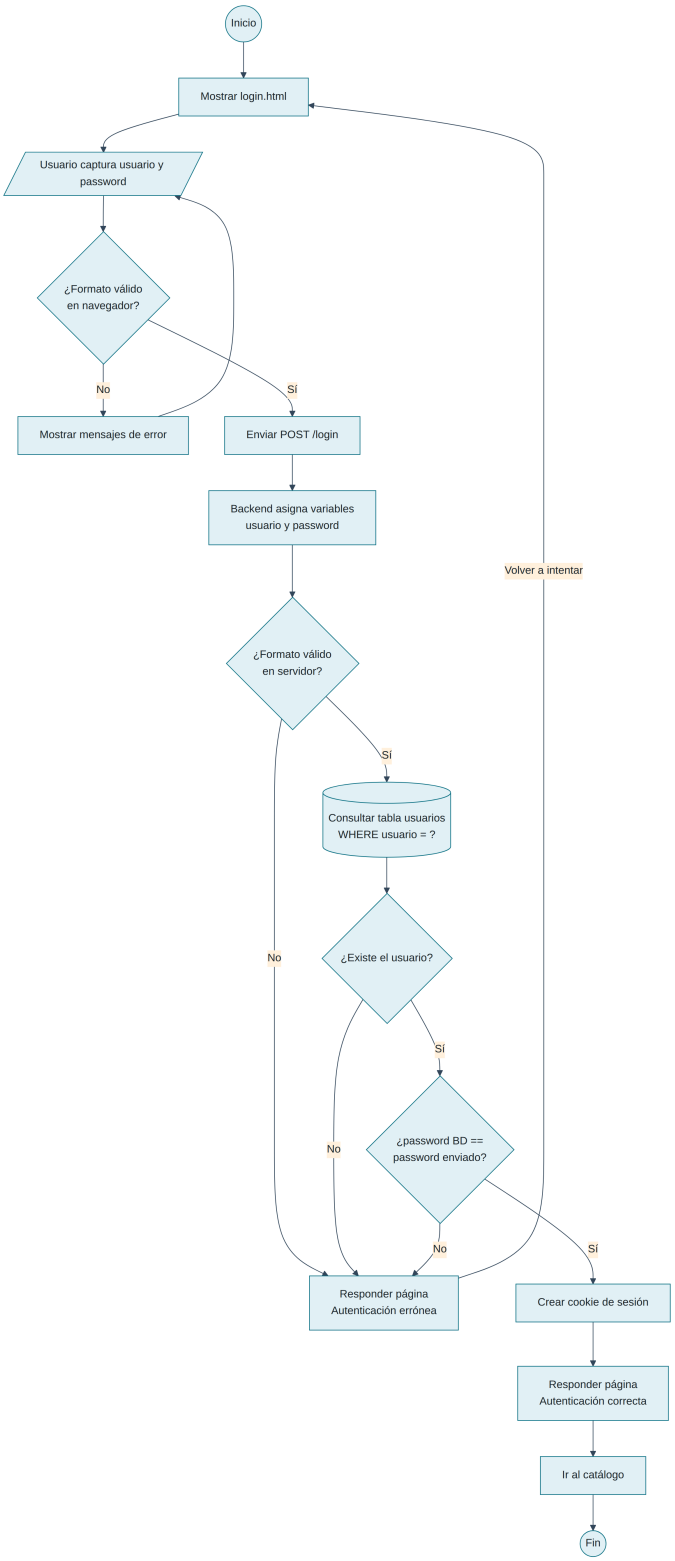


Figura 6. Diagrama de actividad del proceso de autenticación.

El diagrama muestra las dos barreras de validación:

- **Navegador:** evita peticiones innecesarias y da retroalimentación inmediata (campo vacío, longitud, caracteres permitidos, contraseña con letra y número).
- **Servidor:** repite la validación de formato porque una petición puede enviarse sin pasar por el formulario (por ejemplo con `curl`).

Después se consulta la tabla `usuarios` con `WHERE usuario = ?`. Si el usuario no existe o la contraseña no coincide, la respuesta es la misma (“Usuario o contraseña incorrectos”), para no revelar qué dato falló.

Solo cuando ambas condiciones se cumplen se crea la cookie de sesión y se responde la página de autenticación correcta, desde donde se navega al catálogo.

4. Maquetado y diagrama de navegación

4.1 Maquetas (wireframes)

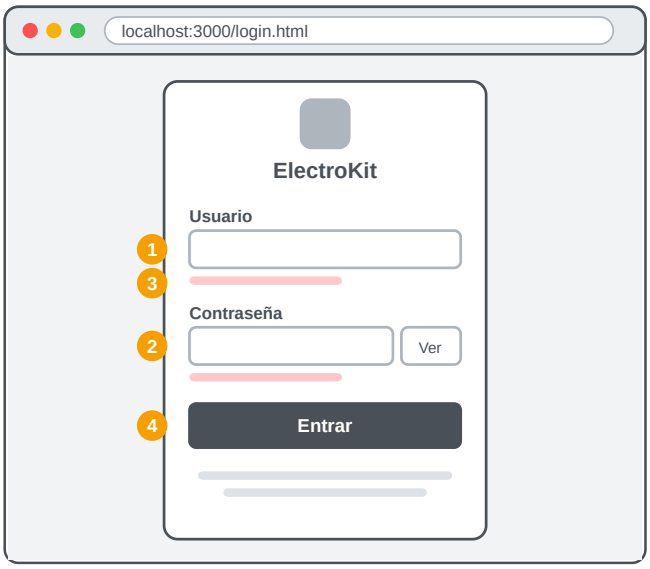


Figura 7. Login. ① Usuario ② Contraseña con botón Ver ③ Mensajes de validación ④ Botón Entrar.



Figura 8. Página de respuesta. ① Ícono ✔/✖ ② Mensaje correcta/errónea ③ Acción siguiente.

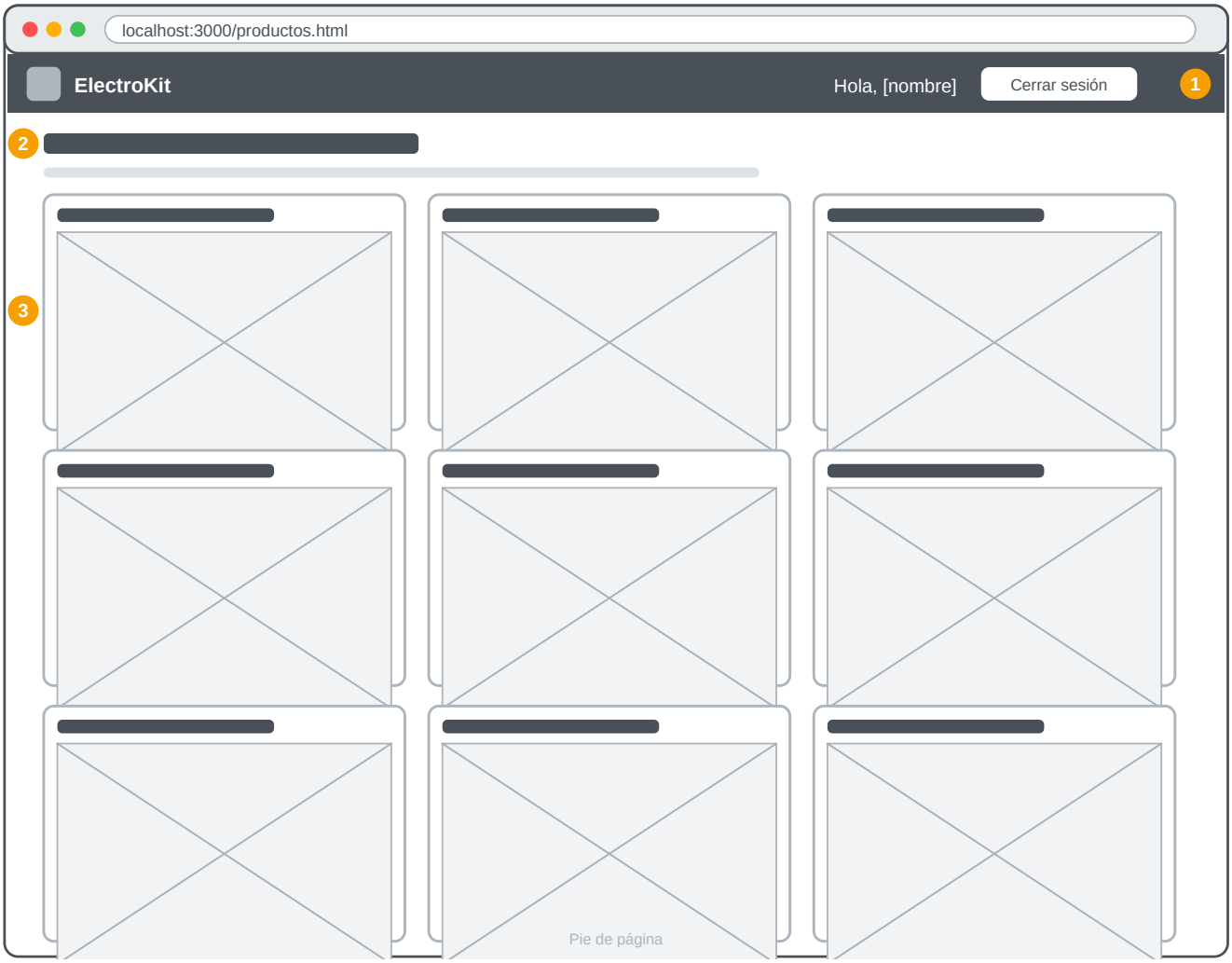


Figura 9. Catálogo. ① Barra con usuario y cerrar sesión ② Título e instrucciones ③ Tabla HTML de 3 columnas × 3 filas.



Figura 10. Detalle de una celda de la tabla.

Elementos de cada celda

#	Elemento
1	Nombre del producto (`<h2>`).
2	Imagen grande (vista principal).
3	5 miniaturas; clic = intercambio con la imagen grande.
4	Descripción.
5	Fecha de envío estimada (hoy + días de envío).
6	Calificación con imágenes `estrella-llena.svg` y `estrella-vacia.svg` .
7	Botón “Comentarios (n)” que muestra u oculta el panel.
8	Lista de comentarios: usuario, estrellas, fecha, texto.
9	Formulario para agregar comentario (calificación + texto).

> **Paleta:** primario `#0b7285`, oscuro `#085565`, acento `#f59f00`, fondo `#eef3f5`, error `#c92a2a`, éxito `#2b8a3e`. Tipografía del sistema (Segoe UI / Roboto / Arial).

4.2 Implementación (capturas de pantalla)

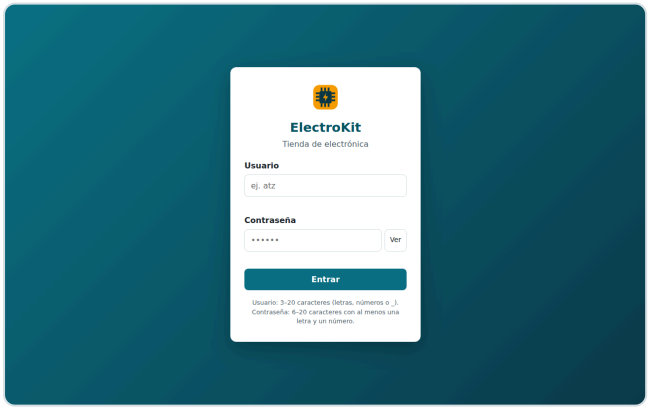


Figura 11. Login.

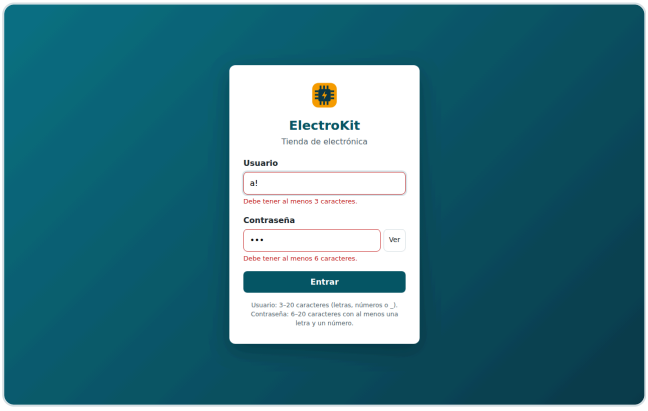


Figura 12. Validación JavaScript.

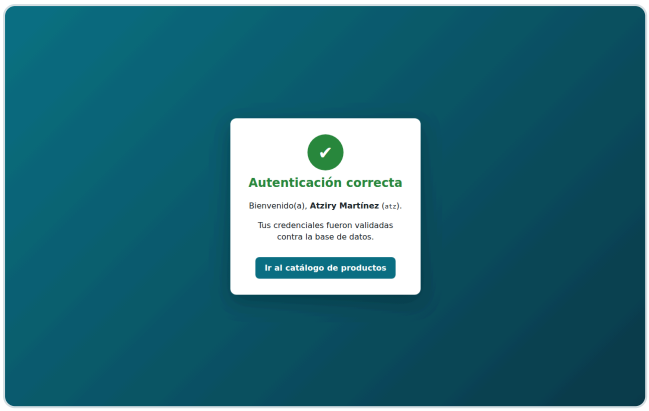


Figura 13. Autenticación correcta.

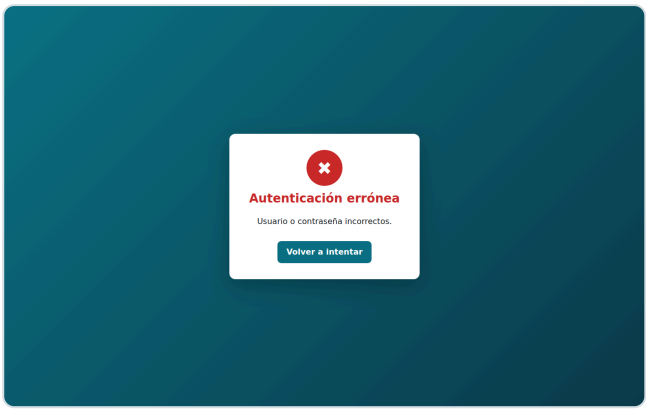


Figura 14. Autenticación errónea.

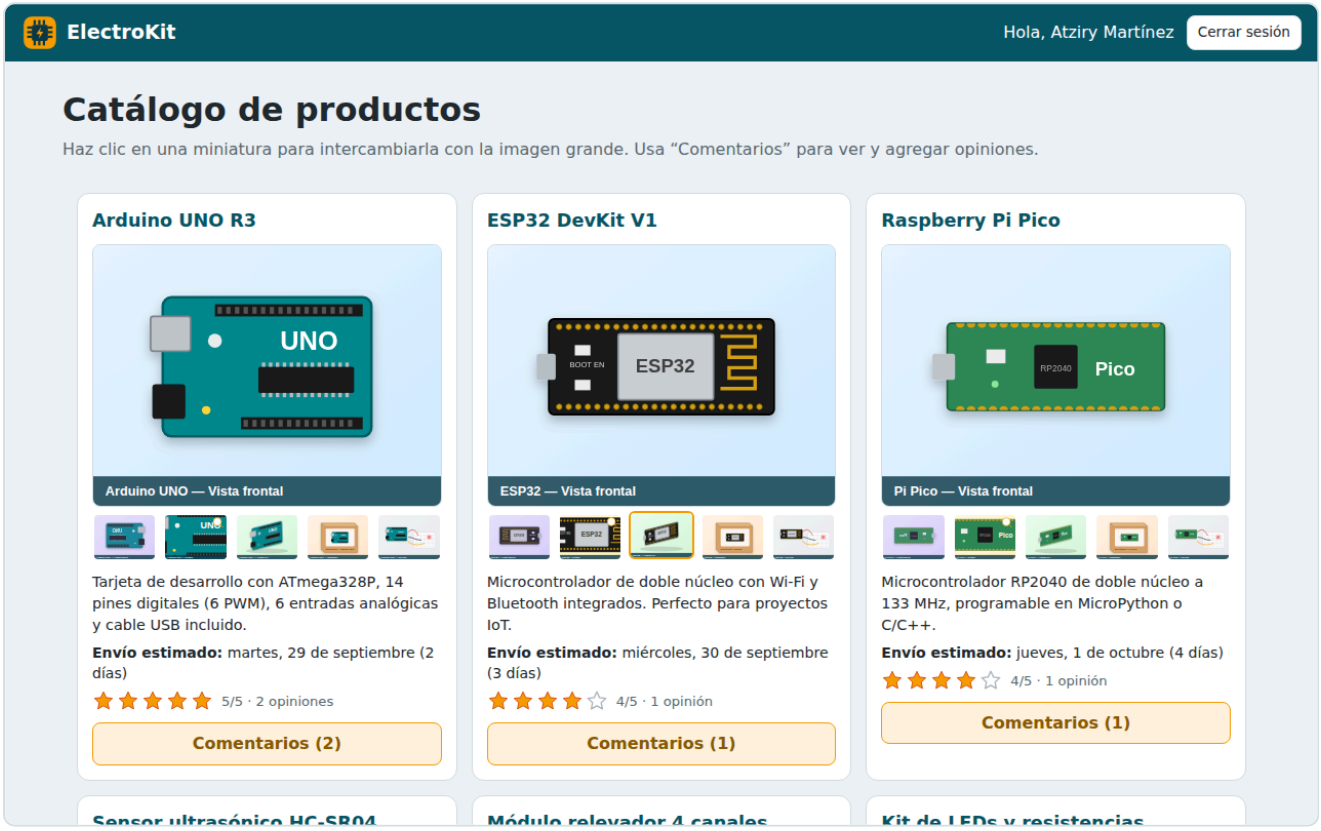


Figura 15. Catálogo: primera fila de la tabla 3×3.

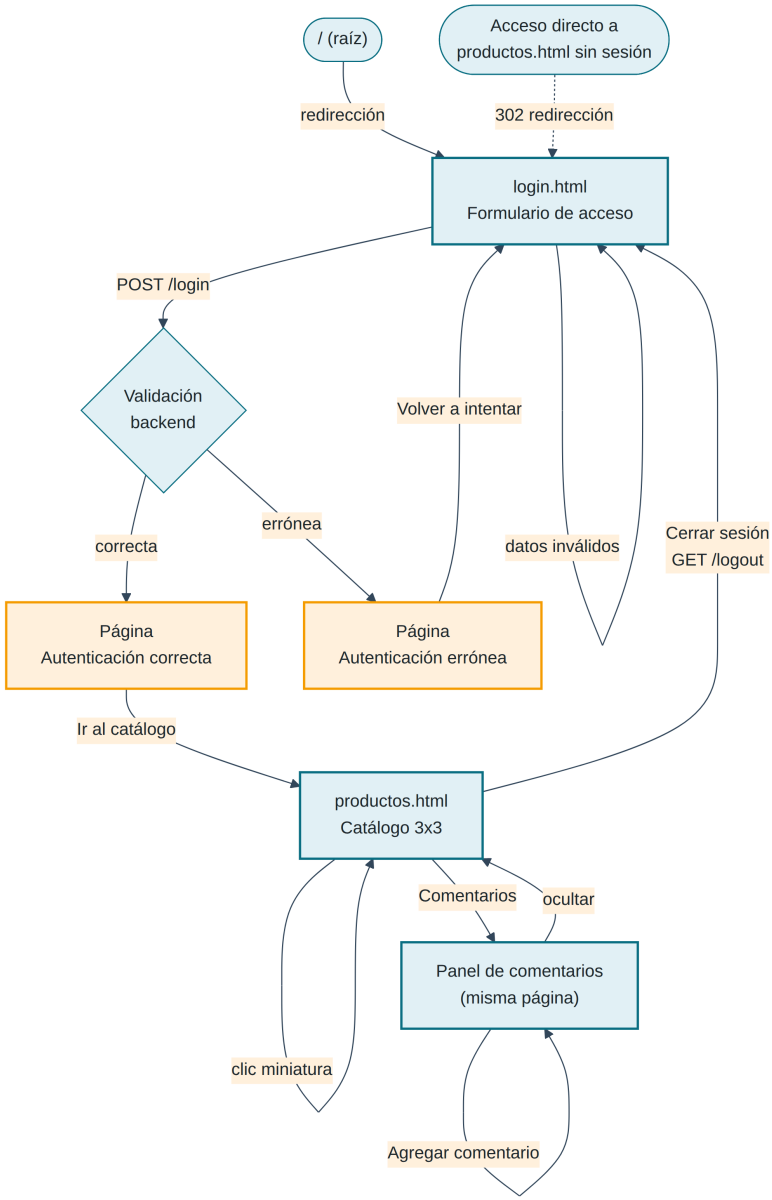


Figura 16. Celda con miniatura intercambiada y comentario agregado.



Figura 17. Vista en celular (390 px).

4.3 Diagrama de navegación



Página / estado	Se llega desde
login.html	Raíz `/`, “Volver a intentar”, “Cerrar sesión” o acceso sin sesión al catálogo.
Autenticación correcta	POST /login con credenciales válidas.
Autenticación errónea	POST /login con usuario inexistente, contraseña distinta o formato inválido.
productos.html	Botón “Ir al catálogo” (requiere cookie de sesión).
Panel de comentarios	Botón “Comentarios” dentro de cada celda (misma página).

Figura 18. Diagrama de navegación entre páginas.

5. Modelo de datos

5.1 Modelo entidad–relación

Conforme al requerimiento, el gestor de base de datos contiene **una sola tabla**, `usuarios`, que almacena las credenciales. No existen relaciones con otras tablas.

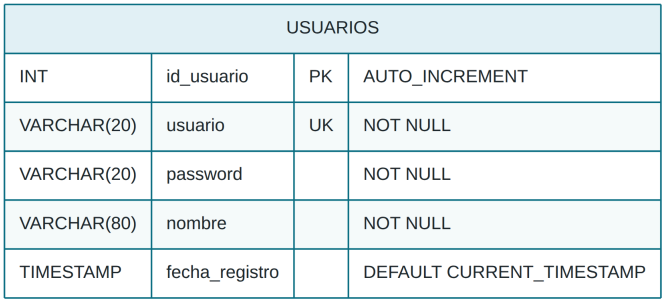


Figura 19. Modelo entidad–relación (base de datos tienda_electronica).

5.2 Diccionario de datos — tabla `usuarios`

Campo	Tipo	Nulo	Llave / restricción	Descripción
`id_usuario`	INT	No	PK, AUTO_INCREMENT	Identificador único del registro.
`usuario`	VARCHAR(20)	No	UNIQUE	Nombre de usuario para iniciar sesión (3–20: letras, números, _).
`password`	VARCHAR(20)	No	—	Contraseña que se compara con la enviada por el navegador.
`nombre`	VARCHAR(80)	No	—	Nombre para mostrar en las páginas.
`fecha_registro`	TIMESTAMP	No	DEFAULT CURRENT_TIMESTAMP	Fecha de alta del usuario.

5.3 Script SQL

```
-- ============================================================
--  Base de datos: tienda_electronica
--  Única tabla requerida: usuarios (usuario y password)
-- ============================================================

SET NAMES utf8mb4;

CREATE DATABASE IF NOT EXISTS tienda_electronica
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE tienda_electronica;

DROP TABLE IF EXISTS usuarios;

CREATE TABLE usuarios (
  id_usuario     INT          NOT NULL AUTO_INCREMENT,
  usuario        VARCHAR(20)  NOT NULL,
  password       VARCHAR(20)  NOT NULL,
  nombre         VARCHAR(80)  NOT NULL,
  fecha_registro TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id_usuario),
  UNIQUE KEY uk_usuarios_usuario (usuario)
) ENGINE=InnoDB;

-- Usuarios de prueba
INSERT INTO usuarios (usuario, password, nombre) VALUES
  ('atz',     'Atz2026',   'Atziry Martínez'),
  ('admin',   'admin123',  'Administrador'),
  ('alumno1', 'upiita01',  'Alumno de prueba');

-- Usuario de aplicación (opcional, recomendado en lugar de root)
-- CREATE USER IF NOT EXISTS 'tienda_app'@'localhost' IDENTIFIED BY 'tienda_pass';
-- GRANT SELECT ON tienda_electronica.usuarios TO 'tienda_app'@'localhost';
-- FLUSH PRIVILEGES;
```

5.4 Modelo de datos del cliente

Los productos y comentarios no se guardan en MySQL (la BD se limita a una sola tabla). Los 9 productos están definidos en `productos-data.js` y los comentarios que agrega el usuario se conservan en el `localStorage` del navegador bajo la clave `electrokit_comentarios`.

```
// Producto
{
  id: "arduino-uno",
  nombre: "Arduino UNO R3",
  descripcion: "Tarjeta de desarrollo ...",
  diasEnvio: 2,          // fecha = hoy + 2
días
  calificacion: 5,       // 1..5 estrellas
  imagenes: [ 6 rutas ], // 1 grande + 5
miniaturas
  comentarios: [ Comentario, ... ]
}
```

```
// Comentario
{
  usuario: "atz",
  estrellas: 4,
  texto: "Excelente tarjeta...",
  fecha: "2026-09-27"
}

// localStorage["electrokit_comentarios"]
{ "arduino-uno": [ Comentario, ... ] }
```

6. Pruebas de funcionamiento

Las pruebas se ejecutaron con el servidor conectado a MariaDB y un navegador Chromium automatizado.

#	Caso de prueba	Entrada	Resultado esperado	Estado
P1	Campos inválidos en el navegador	usuario “a!”, password “abc”	Mensajes de error; el formulario no se envía	✔
P2	Credenciales correctas	atz / Atz2026	200 · “Autenticación correcta”	✔
P3	Contraseña incorrecta	atz / mala123	401 · “Autenticación errónea”	✔
P4	Usuario inexistente	nadie / abc123	401 · “Autenticación errónea”	✔
P5	Intento de inyección SQL	usuario `' OR 1=1 --`	400 · formato inválido (rechazado)	✔
P6	Catálogo sin sesión	GET /productos.html	302 → login.html	✔
P7	Estructura de la tabla	Catálogo con sesión	3 filas × 3 columnas	✔
P8	Intercambio de imágenes	Clic en miniatura	La miniatura pasa a grande y la grande a miniatura	✔
P9	Agregar comentario	4 estrellas + texto	Se agrega a la lista, contador y calificación se actualizan	✔
P10	Vista móvil	Ancho 390 px	Sin desplazamiento horizontal; celdas apiladas	✔

7. Instalación y ejecución

1. Instalar **Node.js 18+** y **MySQL 8** (o MariaDB / XAMPP).
2. Crear la base de datos y la tabla: `mysql -u root -p < database.sql` (o ejecutar el script en MySQL Workbench / phpMyAdmin).
3. En la carpeta del proyecto: `npm install`
4. Configurar credenciales de MySQL si no son las predeterminadas (root sin contraseña). En Windows PowerShell: `$env:DB_PASSWORD="tu_clave"; npm start`. En Linux/macOS: `DB_PASSWORD=tu_clave npm start`
5. Abrir `http://localhost:3000` e iniciar sesión con `atz / Atz2026`, `admin / admin123` o `alumno1 / upiita01`.